

Consistent Hashing

 systemdesignschool.io/fundamentals/consistent-hashing



Consistent hashing is an algorithm used in distributed systems for spreading out data and balancing the load. It was first introduced by David Karger and others in their 1997 paper "[Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web](#)."

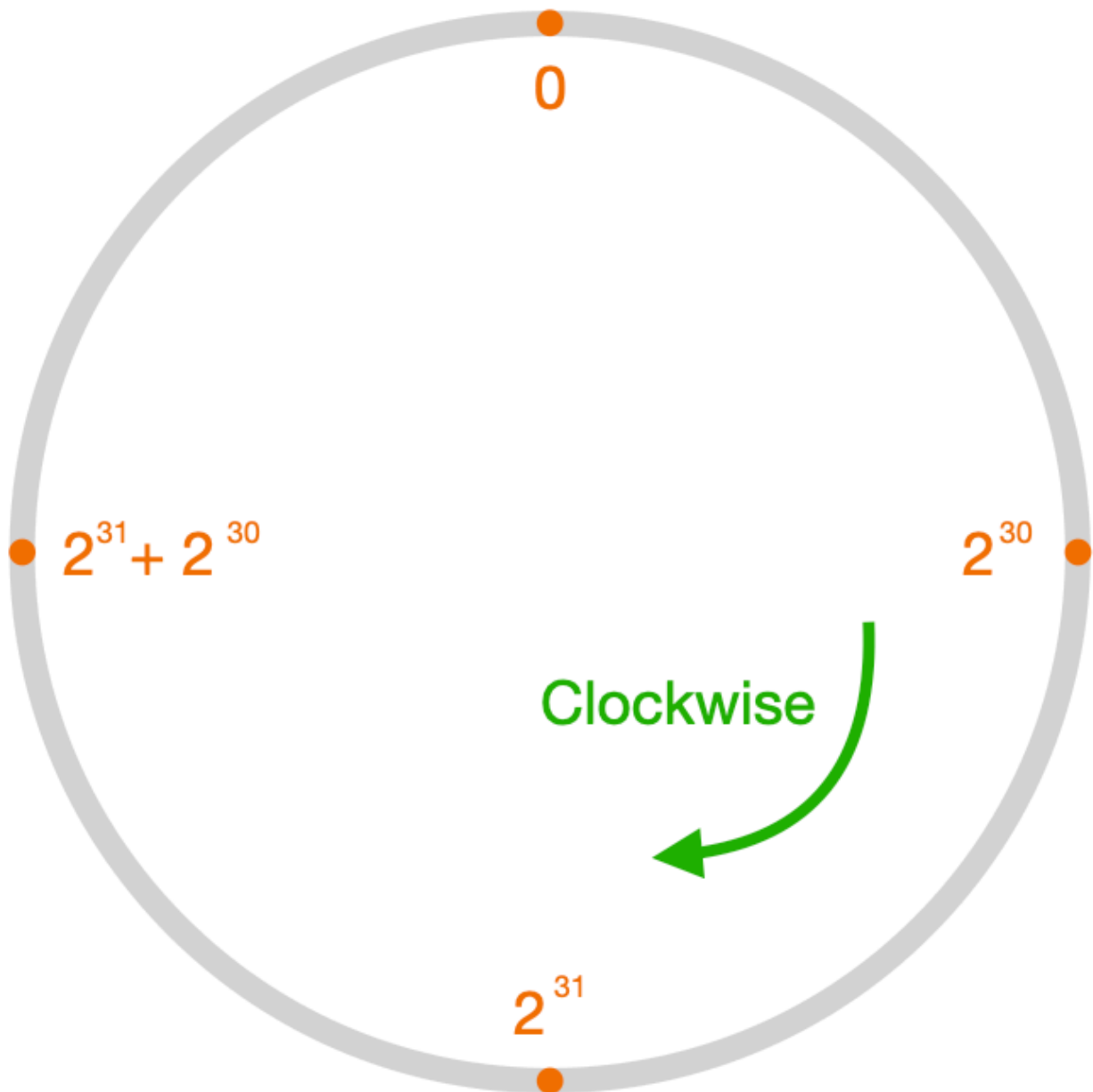
The use of the term "consistent" can be quite confusing. Consistency, in this context, does not refer to [replica consistency](#) or [ACID consistency](#). Here, it describes a particular strategy for maintaining balance. This method is especially useful for caching systems and distributed databases because it minimizes the need to move data around when adding or removing nodes in the system.

In traditional hashing methods, if the size of the hash table changes (like adding or removing storage nodes), almost all key-value pairs need to be remapped to new nodes. This causes a lot of data to be moved and can slow down the system.

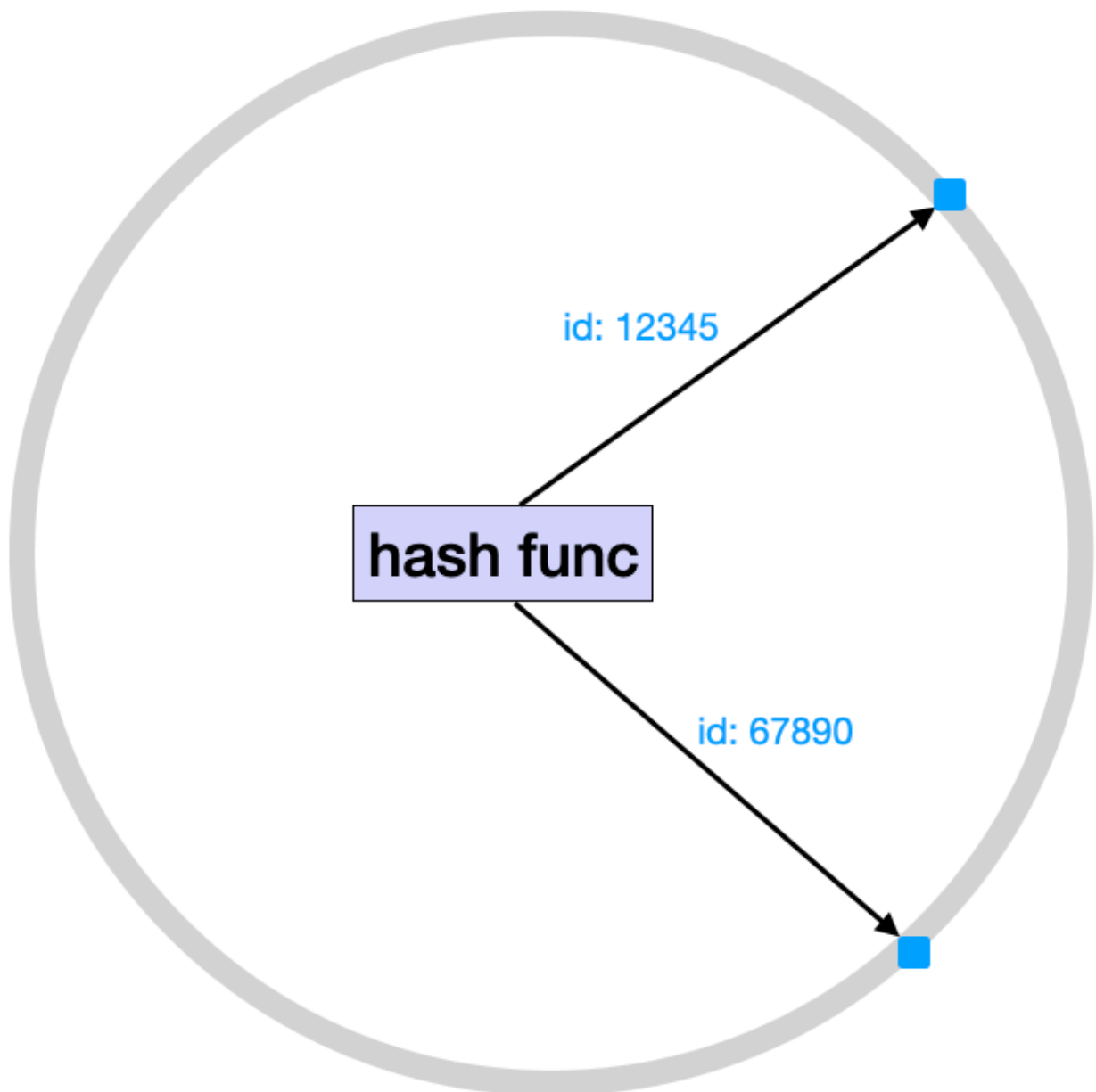
Consistent hashing solves this problem by mapping both data and server nodes onto the same hash ring.

Details of the Algorithm

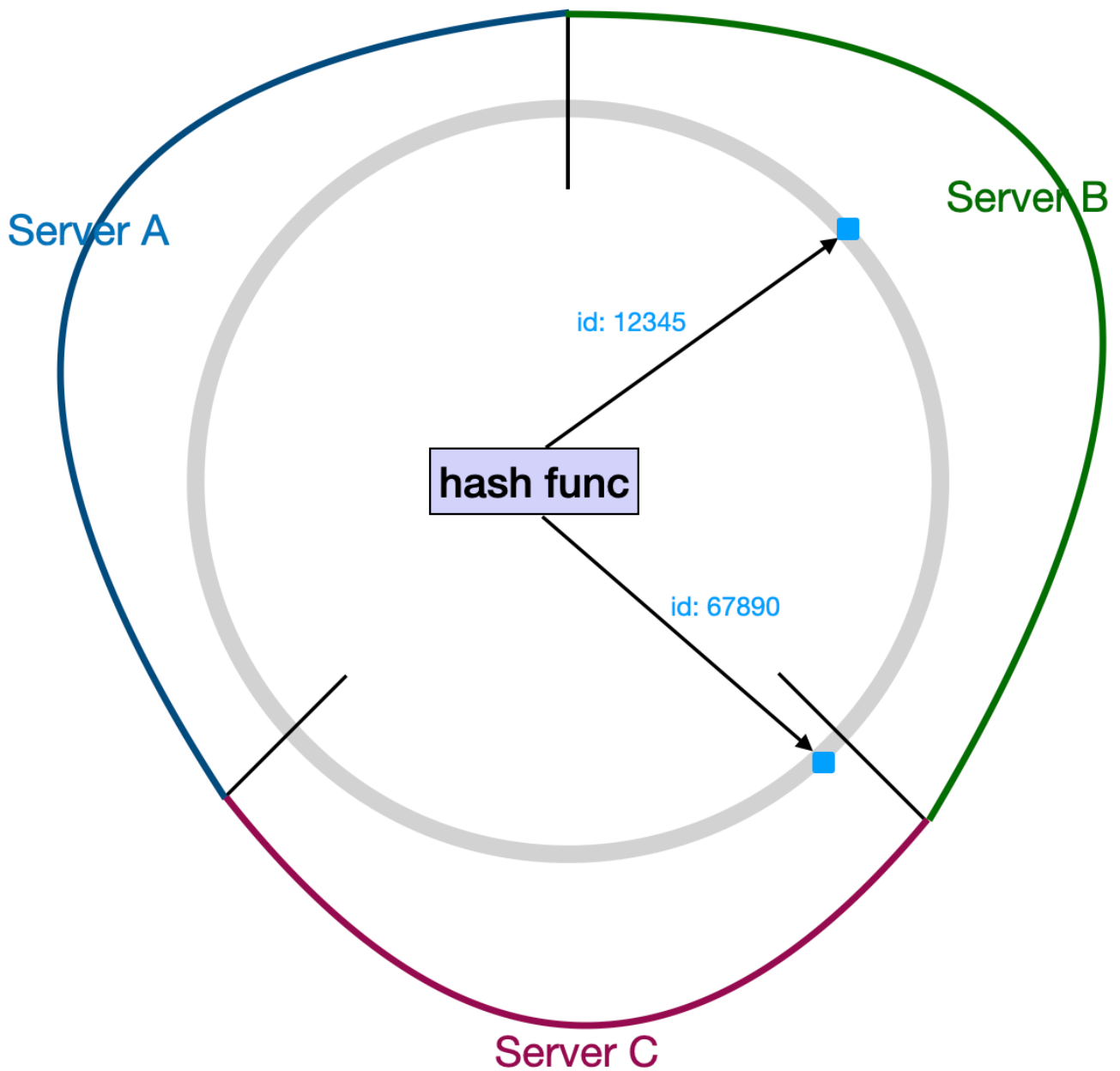
The hash ring is a circle of integers from 0 to $2^{32} - 1$ (assuming a 32-bit hash function is used). Imagine these numbers spread clockwise around a ring:



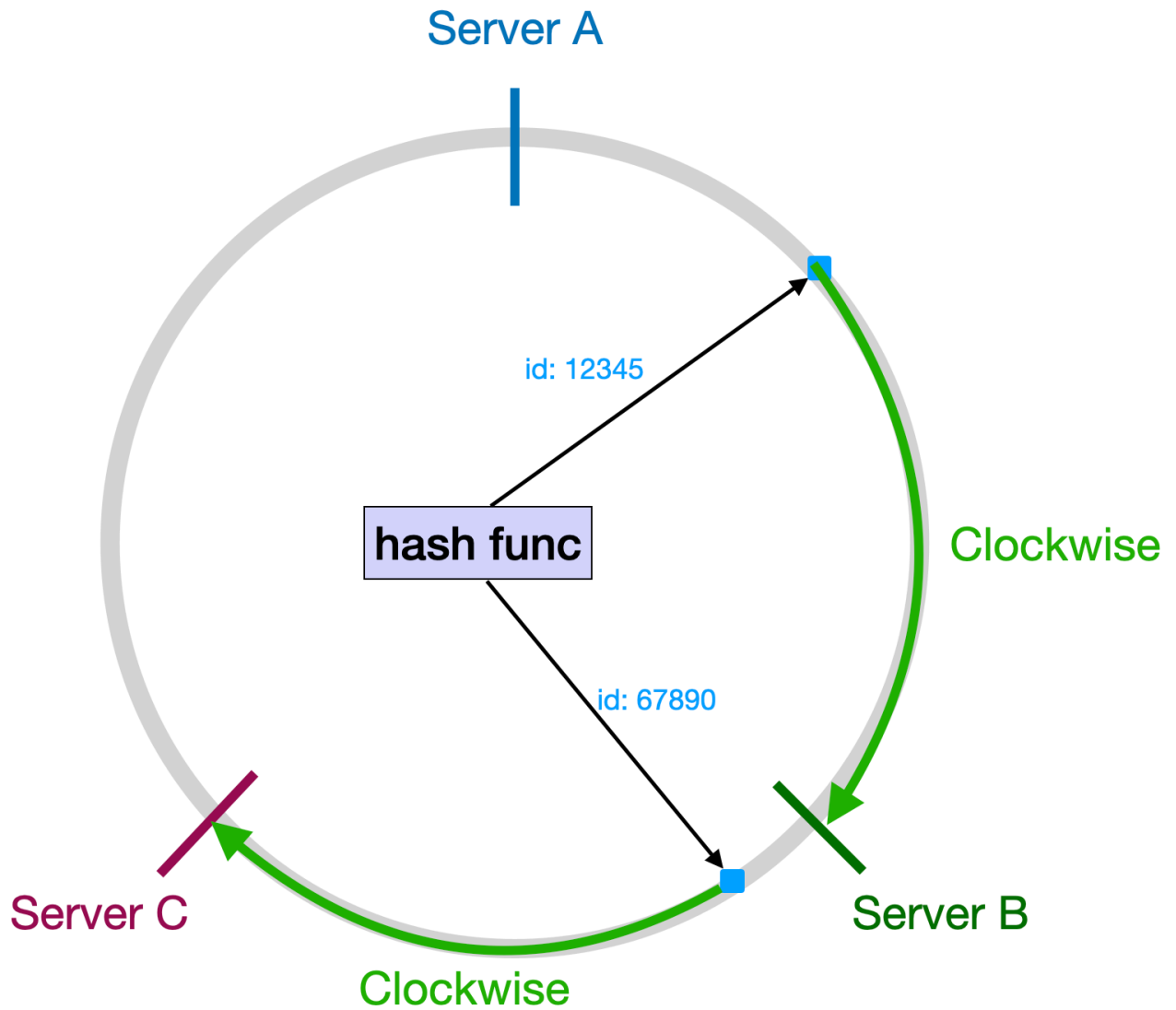
Each piece of data is mapped onto a spot on this ring by a hash function (usually using the data's primary key).



To determine the alignment of data and server nodes, you simply segment the ring into distinct portions corresponding to each server, ensuring no section is overlooked. The area of the ring that a data point falls into after being hashed shows which server it maps to.



Another way to describe it, which is more common and closer to how it's actually implemented, is to place server nodes directly on the ring. The server that corresponds to a data item is the first server point found clockwise from where the data sits on the ring.

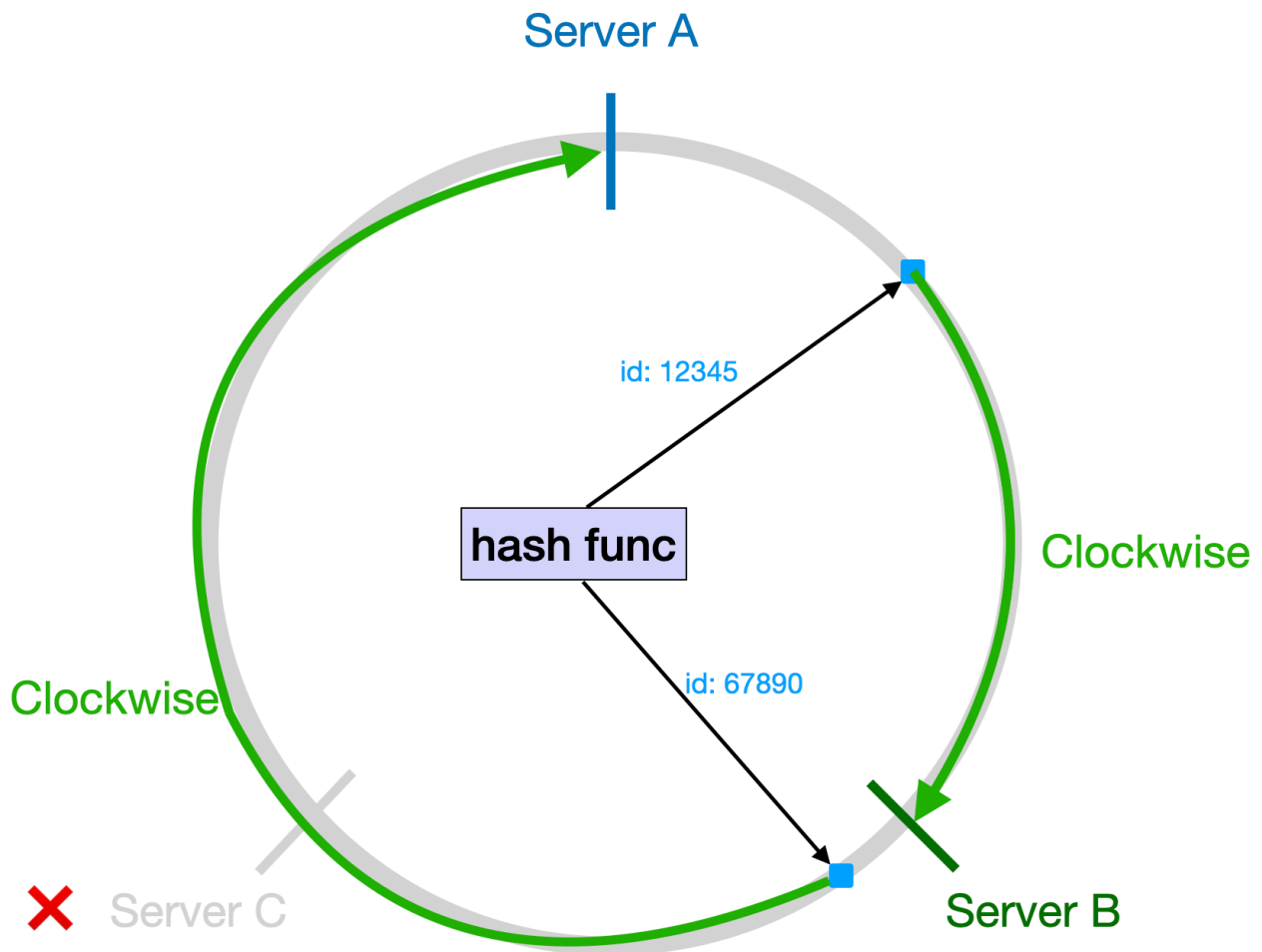


In the image above, Server A, B, and C are three servers and their positions on the ring. The data item with id **12345** finds Server B first going clockwise on the ring, and the data item with id **67890** finds Server C. So, the mapping for the data items is **id:12345 → Server B** and **id:67890 → Server C**, just like before.

The spot for a data item on the ring comes from hashing it. If you think about automating server node mapping, you could hash the node's identifier or IP address. You could also set it manually, considering the server's performance and load, if there aren't many server nodes.

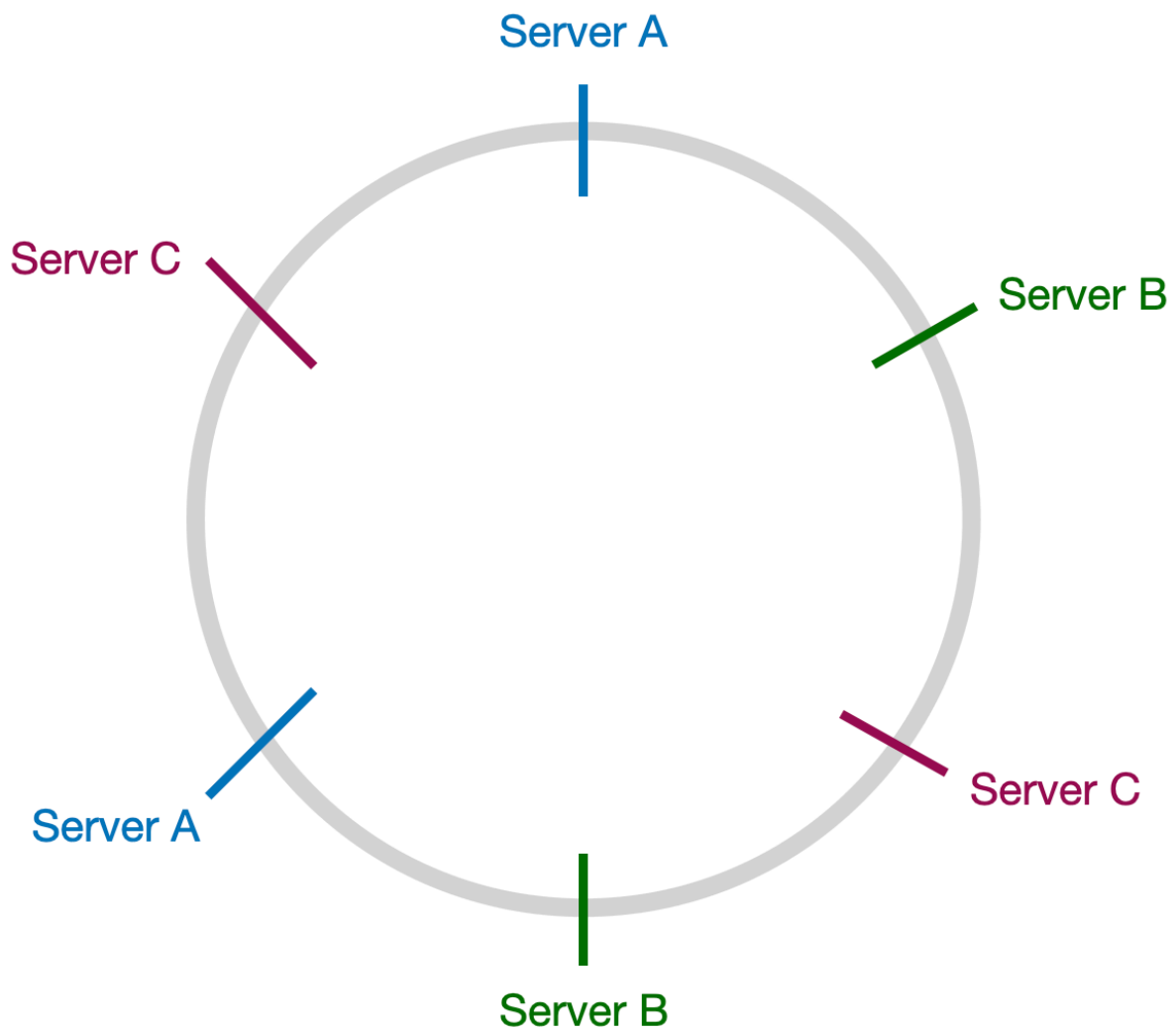
Virtual Nodes

As mentioned earlier, one of the advantages of consistent hashing over traditional hashing algorithms is that when you add or remove a node, only the data items near that node need to be remapped, not all of them. This greatly reduces the amount of data that needs to be moved.



Using the same three servers as an example, if Server C goes down, all of Server C's data has to be moved to Server A, which suddenly increases the load on Server A.

To balance the amount of data and load on each node, the concept of "virtual nodes" is often used. Each physical node maps to several virtual nodes on the hash ring, which helps spread out the data more evenly and makes the system more balanced and stable.



In the image above, each server is mapped to two points on the hash ring. If any one server goes down, its data can be spread across the other two servers.

(Food for thought: If the server nodes on the hash ring are in the order **A-B-C-A-B-C**, would it still work the same way?)

Overall, consistent hashing is a technique that provides good load balancing and minimizes data movement when nodes are added or removed. It's very practical for designing distributed systems.